That list immediately eliminates many bad choices. A text-only reasoning model may be excellent for code but useless if your main requirement is analysing screenshots. A giant multimodal model may be unnecessary if you mostly want fast rewriting. A specialist translation model may beat a general chatbot for language conversion but be hopeless as an all-purpose assistant.



Most readers will fall into five broad categories.

General-purpose assistant: writing, summarisation, brainstorming, question answering and routine productivity.

Reasoning: maths, logic, structured problem solving and difficult multi-step questions.

Coding: generating, explaining and debugging software.

Vision: understanding screenshots, photographs, charts and scanned documents.

Indian-language work: Hindi, Hinglish or regional-language conversation, translation and culturally specific queries.

You may need more than one. In fact, by the end of this Fast Track, having two or three local models for different jobs is often smarter than forcing one heavyweight model to do everything.

Step 2: Decide on generalist or specialist

General-purpose models are the safest starting point. They can usually write, summarise, answer questions and perform some coding and reasoning. Current families also increasingly combine modalities and reasoning modes.

Google's Gemma 4 family, for instance, ranges from E2B and E4B models through 12B, 26B-A4B and 31B versions. Google lists text and image input across the family, audio input on several sizes, multilingual support covering more than 140 languages and context windows reaching 256K tokens on some models. That makes the family useful precisely because you can choose different capability and size points without leaving the ecosystem.

Qwen provides another broad ladder. Its 2026 releases include small Qwen3.5 variants at 0.8B, 2B, 4B and 9B, while the later Qwen3.6-35B-A3B is a multimodal MoE model with 35B total parameters and around 3B active. The Qwen3.6 model card lists Apache 2.0 licensing and compatibility with Transformers, vLLM, SGLang and other runtimes.

A specialist becomes attractive when one task dominates. Qwen3-Coder, for example, is explicitly built around coding and agentic software work, with variants including a 30B-A3B model and Qwen3-Coder-Next. If 80 percent of your local-AI use is programming, a coding specialist deserves a place on your shortlist even if a general model scores well elsewhere.

The rule is simple: choose breadth when your workload is broad; choose specialisation when your workload is repetitive.

Step 2½: Prefer quality per gigabyte, not prestige

Local AI introduces a metric that cloud users rarely have to think about: how much useful capability you get for every gigabyte you must store and load.

A model that is marginally better on a benchmark but twice the size may be a poor local choice. Larger models take longer to download, occupy more SSD space, consume more RAM or VRAM and can generate tokens more slowly on the same machine. That matters because local AI is something you will actually wait for.

This is where smaller current-generation models deserve serious attention. A well-trained 4B or 9B model can be far more satisfying for everyday work than an older, much larger model running at a crawl. Conversely, do not fetishise tiny models simply because they are fast. If you constantly correct their mistakes, the seconds saved during generation disappear during editing.

Try thinking in terms of a "usefulness budget". Suppose Model A produces acceptable answers in three seconds using 6GB of memory, while Model B produces slightly better answers in 18 seconds and uses 14GB. For routine rewriting, A may be the sensible default. For an important reasoning task, B may justify the wait.

This is also why there is no shame in keeping several models installed. Local storage is comparatively cheap; your time is not. Choose a lightweight default and call on heavier models only when the task earns the extra computation.

Step 3: Put a ceiling on model size

Now bring your hardware into the conversation.

You learned in the previous chapter that parameter count, precision and quantisation influence memory use. Apply that knowledge before you become attached to a model.